Estimación del tráfico de red en un servidor web mediante la regla del trapecio compuesto

Total de datos transferidos por un microservicio durante un pico de demanda de 12 segundos

Estudiante: **Fernando Istaña Flores** Escuela: **Ingeniería de Sistemas**

1. Contexto y datos

En un centro de procesamiento de datos, el consumo de ancho de banda de un microservicio crítico de procesamiento de transacciones no es constante: fluctúa durante un pico de demanda de 12 segundos. El equipo de infraestructura necesita conocer la cantidad total de datos transferidos D (en Megabits) en ese intervalo, que es el área bajo la curva de la tasa de transferencia R(t) (en Mbps):

$$D = \int_0^{12} R(t)\,dt, \qquad R(t) = 10 + 5\cdot\operatorname{sen}(\pi t/4) + e^{0.15t}$$

La tasa tiene tres componentes: una carga base constante de 10 Mbps, una oscilación de amplitud 5 Mbps y periodo 8 s (el término senoidal) y una tendencia creciente exponencial. El seno se evalúa en **radianes**. Se pide aproximar la integral con la **regla del trapecio compuesto con n = 4 subintervalos**, automatizar el cálculo para cualquier n y estudiar cómo converge.

2. Construcción manual — Trapecio compuesto con n = 4

La regla del trapecio compuesto divide [a, b] en n subintervalos de igual ancho, aproxima la curva con un segmento de recta en cada uno y suma las áreas de los trapecios resultantes:

$$I \approx \frac{h}{2}\cdot\left[\, f(x_0) + 2\cdot\left(f(x_1) + f(x_2) + \dots + f(x_{n-1}) \right) + f(x_n) \,\right], \qquad h = \frac{b-a}{n}$$

Los extremos aparecen una sola vez y los nodos interiores dos veces, porque cada uno es el lado derecho de un trapecio y el lado izquierdo del siguiente.

Paso 1 — Ancho de paso

$$h = \frac{b-a}{n} = \frac{12-0}{4} = 3 \text{ s}$$

Paso 2 — Malla de puntos y valores de R(tᵢ)

$$t_i = a + i\cdot h = 3i \quad\rightarrow\quad t_0 = 0,\ t_1 = 3,\ t_2 = 6,\ t_3 = 9,\ t_4 = 12$$

i	t_i (s)	$5\cdot\operatorname{sen}(\pi t_i/4)$	$e^{0.15t_i}$	$R(t_i)$ (Mbps)	Peso	Peso·$R(t_i)$
0	0	0.000000	1.000000	11.000000	1	11.000000
1	3	3.535534	1.568312	15.103846	2	30.207692
2	6	−5.000000	2.459603	7.459603	2	14.919206
3	9	3.535534	3.857426	17.392959	2	34.785919
4	12	0.000000	6.049647	16.049647	1	16.049647

Valores del seno: sen(0) = 0, sen(3π/4) = √2/2 ≈ 0.707107, sen(3π/2) = −1, sen(9π/4) = sen(π/4) = √2/2 y sen(3π) = 0.

Paso 3 — Aplicación de la fórmula

$$\begin{aligned} I &\approx \frac{3}{2}\cdot\left[R(0) + 2\cdot\left(R(3) + R(6) + R(9) \right) + R(12) \right] \\ &= 1.5\cdot\left[11.000000 + 2\cdot\left(15.103846 + 7.459603 + 17.392959 \right) + 16.049647 \right] \\ &= 1.5\cdot\left[11.000000 + 2\cdot\left(39.956408 \right) + 16.049647 \right] \\ &= 1.5\cdot\left(106.962463 \right) \\ &= 160.443695 \;\approx\; 160.4437 \text{ Mb} \end{aligned}$$

I_trapecio (n = 4) ≈ 160.4437 Mb

Arrastrando 6 decimales a mano, el último dígito puede diferir del programa (160.443697) por redondeo; no afecta al resultado redondeado a 4 decimales.

Paso 4 — Valor exacto para contrastar

La función tiene primitiva elemental, de modo que la integral se puede calcular de forma analítica:

```
D = [ 10t − (20/π)·cos(πt/4) + e^(0.15t)/0.15 ]₀¹²
  = 10(12) − (20/π)·[cos(3π) − cos(0)] + (e^1.8 − e⁰)/0.15
  = 120 − (20/π)·(−1 − 1) + (6.049647 − 1)/0.15
  = 120 + 40/π + 33.664316
  = 120 + 12.732395 + 33.664316 = 166.396712 Mb

Error absoluto  = |166.396712 − 160.443697| = 5.953015 Mb
Error relativo  = 5.953015 / 166.396712 · 100 % = 3.5776 %
```

3. Desarrollo manual

[Aquí va la foto de la hoja con el desarrollo a mano: ancho de paso, malla, valores de $R(t_i)$ y fórmula aplicada.]

4. Resultado

D ≈ 160.4437 Mb con n = 4. Con cuatro subintervalos, el trapecio compuesto estima que el microservicio transfirió unos 160.44 Megabits durante el pico de 12 segundos, unos 20 MB. El valor exacto es 166.3967 Mb: la aproximación lo subestima en 5.95 Mb (3.58 %).

La subestimación se explica en la gráfica de la sección 5: entre t = 0 y t = 3 y entre t = 9 y t = 12 la curva forma «jorobas» por encima de las cuerdas (allí R es cóncava, R'' < 0) y esas áreas quedan fuera de los trapecios. En el valle de t = 6 ocurre lo contrario, pero el área que sobra es menor que la que falta.

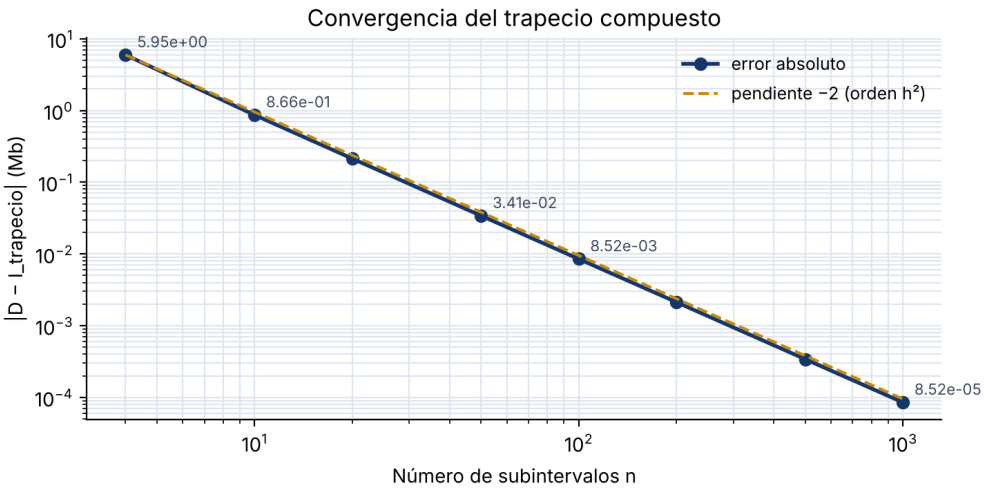
5. Aplicativo solucionador

El programa implementa la función trapecio_compuesto(f, a, b, n), que calcula la integral para cualquier número de subintervalos: obtiene h = (b − a)/n, suma los nodos interiores, les añade la media de los extremos y multiplica por h. Con ella se reproduce el caso n = 4 y se construye la tabla de convergencia. La versión web hace el mismo cálculo mientras se escribe, dibuja los trapecios sobre la curva y estima además la cota teórica del error.

Análisis de convergencia

n	h (s)	I_trapecio (Mb)	Error absoluto (Mb)	Error %	Razón de errores
4	**3**	**160.443697**	**5.953015**	**3.577604**	**—**
10	1.2	165.530824	0.865888	0.520376	6.88
50	0.24	166.362626	0.034086	0.020485	25.40
100	0.12	166.388195	0.008517	0.005119	4.00
1000	0.012	166.396627	8.5159×10^{-5}	0.000051	100.02

La razón de errores confirma que el trapecio es un método de **orden h^2**: al pasar de n = 4 a n = 10 (2.5 veces más subintervalos) el error baja 6.88 ≈ 2.5^2 veces; de 50 a 100 baja 4.00 = 2^2 veces, y de 100 a 1000 baja 100.02 ≈ 10^2 veces. Con n = 1000 el error ya es de 8.5×10^{-5} Mb.



Error absoluto contra n en escala logarítmica: los puntos siguen una recta de pendiente −2, la huella del orden h^2.

Motor de cálculo

```
"""
Análisis Numérico · Unidad 3 · Sesión 12
Integración numérica: regla del trapecio compuesto. Tráfico de red de un microservicio.

La tasa de transferencia (Mbps) durante un pico de 12 segundos es
    R(t) = 10 + 5·sen(pi·t/4) + e^(0.15·t)
y el total de datos transferidos (Megabits) es
    D = integral de 0 a 12 de R(t) dt

Fernando Istaña Flores · Universidad Peruana Unión
"""

import math


# ---------------------------------------------------------------------------
# Modelo
# ---------------------------------------------------------------------------
def R(t):
    """Tasa de transferencia en Mbps (el seno trabaja en radianes)."""
    return 10 + 5 * math.sin(math.pi * t / 4) + math.exp(0.15 * t)


def integral_exacta(a=0.0, b=12.0):
    """Primitiva: 10t − (20/pi)·cos(pi·t/4) + e^(0.15t)/0.15."""
    F = lambda t: 10 * t - (20 / math.pi) * math.cos(math.pi * t / 4) + math.exp(0.15 * t) / 0.15
    return F(b) - F(a)


# ---------------------------------------------------------------------------
# Módulo de cálculo
# ---------------------------------------------------------------------------
def trapecio_compuesto(f, a, b, n):
    """
    Regla del trapecio compuesto con n subintervalos de igual ancho.

        h = (b − a) / n
        I ≈ (h/2)·[ f(x0) + 2·(f(x1) + … + f(x_{n−1})) + f(xn) ]

    Los extremos pesan 1 y los nodos interiores pesan 2.
    """
    if n < 1:
```

```
        raise ValueError("n debe ser un entero positivo.")
    h = (b - a) / n
    interior = sum(f(a + i * h) for i in range(1, n))
    return h / 2 * (f(a) + 2 * interior + f(b))


def malla(f, a, b, n):
    """Lista de (i, t_i, f(t_i), peso) para mostrar el cálculo a mano."""
    h = (b - a) / n
    return [(i, a + i * h, f(a + i * h), 1 if i in (0, n) else 2) for i in range(n + 1)]


if __name__ == "__main__":
    a, b = 0.0, 12.0
    exacta = integral_exacta(a, b)

    print("=" * 74)
    print("PARTE 1 · TRAPECIO COMPUESTO CON n = 4")
    print("=" * 74)
    n = 4
    h = (b - a) / n
    print("  h = (b − a)/n = (%g − %g)/%d = %g s" % (b, a, n, h))
    print()
    print("   i     t_i     5·sen(pi·t/4)    e^(0.15t)        R(t_i)   peso   peso·R")
    suma = 0.0
    for i, t, y, w in malla(R, a, b, n):
        s = 5 * math.sin(math.pi * t / 4)
        e = math.exp(0.15 * t)
        suma += w * y
        print("  %2d  %6.2f   %13.6f   %10.6f   %12.6f   %4d   %10.6f" % (i, t, s + 0.0, e, y, w, w *
y))
    I4 = trapecio_compuesto(R, a, b, n)
    print()
    print("  Suma ponderada = %.6f" % suma)
    print("  I_trapecio = (h/2)·suma = (%g/2)·%.6f = %.6f Mb" % (h, suma, I4))

    print()
    print("=" * 74)
    print("VALOR EXACTO (integración analítica)")
    print("=" * 74)
    print("  D = [10t − (20/pi)·cos(pi·t/4) + e^(0.15t)/0.15] de 0 a 12")
    print("    = 120 + 40/pi + (e^1.8 − 1)/0.15")
    print("    = 120 + %.6f + %.6f = %.6f Mb" % (40 / math.pi, (math.exp(1.8) - 1) / 0.15, exacta))

    print()
    print("=" * 74)
    print("PARTE 2 · ANÁLISIS DE CONVERGENCIA")
    print("=" * 74)
    print("  %6s %10s %16s %16s %12s %10s" % ("n", "h", "I_trapecio", "error absoluto", "error %",
"razón"))
    previo = None
    filas = []
    for n in (4, 10, 50, 100, 1000):
        I = trapecio_compuesto(R, a, b, n)
        ea = abs(exacta - I)
        er = ea / abs(exacta) * 100
        razon = "" if previo is None else "%.2f" % (previo[1] / ea)
        print("  %6d %10.4f %16.6f %16.6e %12.6f %10s" % (n, (b - a) / n, I, ea, er, razon))
        filas.append((n, I, ea, er))
        previo = (n, ea)
    print()
    print("  'razón' = error anterior / error actual. El trapecio es de orden h²:")
```

```
    print("  si n se multiplica por k, el error se divide aproximadamente entre k².")
    print("  De 100 a 1000 (k = 10) el error baja unas %.0f veces." % (filas[3][2] / filas[4][2]))

    # ------------------------------------------------------------------------
    # Gráficas: la función con los 4 trapecios, y el error contra n
    # ------------------------------------------------------------------------
    try:
        import matplotlib
        matplotlib.use("Agg")
        import matplotlib.pyplot as plt

        t = [a + (b - a) * k / 600 for k in range(601)]
        fig, ax = plt.subplots(figsize=(7.4, 4.2), dpi=200)
        pts = malla(R, a, b, 4)
        for i in range(4):
            x0, y0 = pts[i][1], pts[i][2]
            x1, y1 = pts[i + 1][1], pts[i + 1][2]
            ax.fill([x0, x0, x1, x1], [0, y0, y1, 0], color="#c8890a", alpha=0.18,
                    edgecolor="#c8890a", linewidth=1.2, label="trapecios (n = 4)" if i == 0 else None)
        ax.fill_between(t, [R(v) for v in t], color="#14366b", alpha=0.06)
        ax.plot(t, [R(v) for v in t], color="#14366b", linewidth=2, label="R(t) = 10 + 5·sen(πt/4) +
e^(0.15t)")
        ax.scatter([p[1] for p in pts], [p[2] for p in pts], color="#101c33", zorder=3, s=36,
label="nodos tᵢ")
        for p in pts:
            ax.annotate("%.3f" % p[2], (p[1], p[2]), textcoords="offset points", xytext=(4, 7),
fontsize=8, color="#3c4a63")
        ax.set_xlabel("Tiempo t (s)")
        ax.set_ylabel("Tasa R(t) (Mbps)")
        ax.set_title("Trapecio compuesto, n = 4: %.4f Mb  ·  exacto: %.4f Mb" % (I4, exacta))
        ax.set_ylim(0, 21)
        ax.grid(color="#dfe5ef", linewidth=0.8)
        ax.set_axisbelow(True)
        for lado in ("top", "right"):
            ax.spines[lado].set_visible(False)
        ax.legend(frameon=False, fontsize=8, loc="lower right")
        fig.tight_layout()
        fig.savefig("trapecio_s12.png")

        fig, ax = plt.subplots(figsize=(7.4, 3.8), dpi=200)
        ns = [4, 10, 20, 50, 100, 200, 500, 1000]
        errs = [abs(exacta - trapecio_compuesto(R, a, b, k)) for k in ns]
        ax.loglog(ns, errs, "o-", color="#14366b", linewidth=2, label="error absoluto")
        ax.loglog(ns, [errs[0] * (4 / k) ** 2 for k in ns], "--", color="#c8890a", label="pendiente −2
(orden h²)")
        for k, e in zip(ns, errs):
            if k in (4, 10, 50, 100, 1000):
                ax.annotate("%.2e" % e, (k, e), textcoords="offset points", xytext=(6, 4), fontsize=8,
color="#3c4a63")
        ax.set_xlabel("Número de subintervalos n")
        ax.set_ylabel("|D − I_trapecio| (Mb)")
        ax.set_title("Convergencia del trapecio compuesto")
        ax.grid(color="#dfe5ef", linewidth=0.8, which="both")
        for lado in ("top", "right"):
            ax.spines[lado].set_visible(False)
        ax.legend(frameon=False, fontsize=9)
        fig.tight_layout()
        fig.savefig("convergencia_s12.png")
        print()
        print("  Gráficas guardadas en trapecio_s12.png y convergencia_s12.png")
    except ImportError:
        print()
```

```
        print("  (matplotlib no está instalado: se omitieron las gráficas)")
```

Salida del programa

```
================================================================================
PARTE 1 · TRAPECIO COMPUESTO CON n = 4
================================================================================
  h = (b − a)/n = (12 − 0)/4 = 3 s

   i     t_i     5·sen(pi·t/4)    e^(0.15t)        R(t_i)   peso   peso·R
   0    0.00         0.000000     1.000000      11.000000      1    11.000000
   1    3.00         3.535534     1.568312      15.103846      2    30.207692
   2    6.00        -5.000000     2.459603       7.459603      2    14.919206
   3    9.00         3.535534     3.857426      17.392959      2    34.785919
   4   12.00         0.000000     6.049647      16.049647      1    16.049647

  Suma ponderada = 106.962465
  I_trapecio = (h/2)·suma = (3/2)·106.962465 = 160.443697 Mb

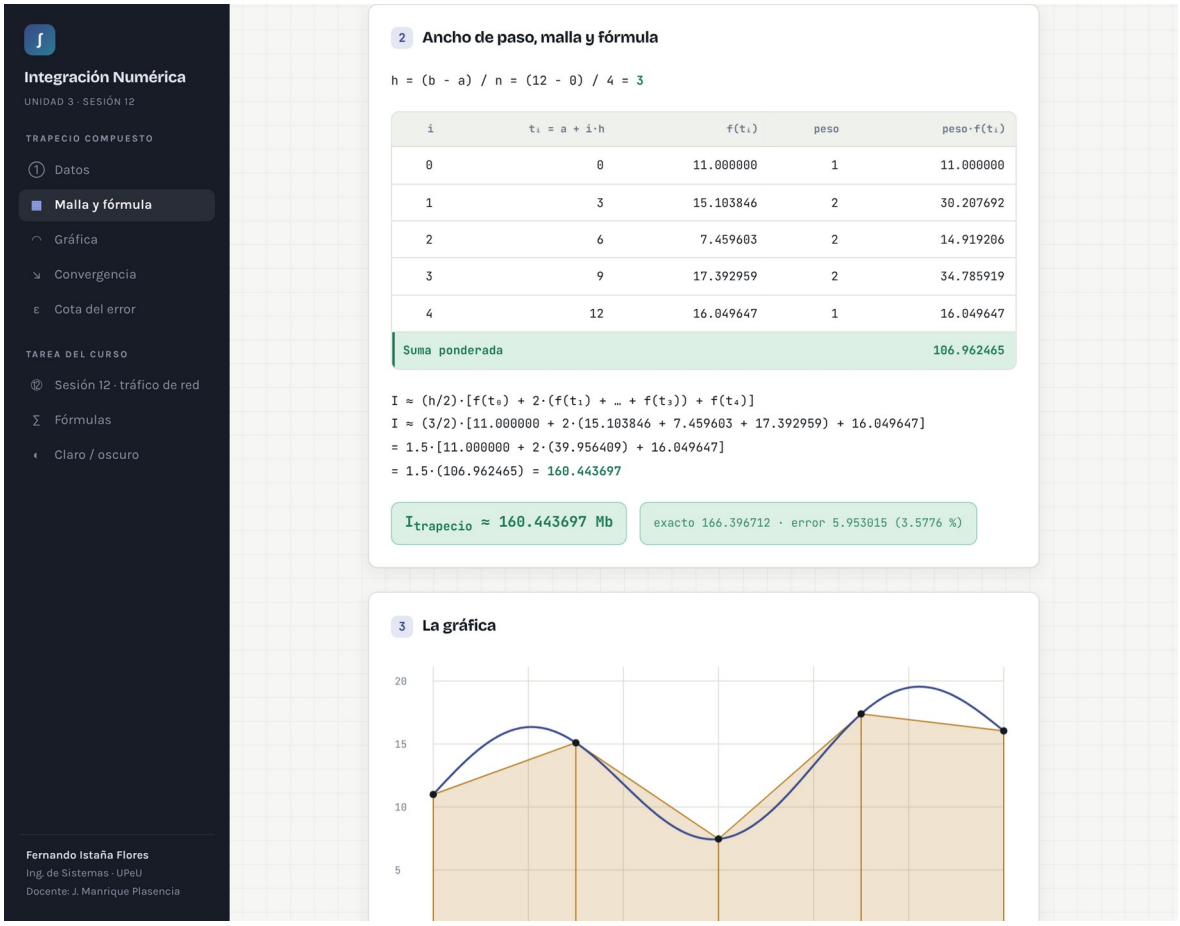
================================================================================
VALOR EXACTO (integración analítica)
================================================================================
  D = [10t − (20/pi)·cos(pi·t/4) + e^(0.15t)/0.15] de 0 a 12
    = 120 + 40/pi + (e^1.8 − 1)/0.15
    = 120 + 12.732395 + 33.664316 = 166.396712 Mb

================================================================================
PARTE 2 · ANÁLISIS DE CONVERGENCIA
================================================================================
       n          h       I_trapecio   error absoluto      error %      razón
       4     3.0000       160.443697     5.953015e+00     3.577604
      10     1.2000       165.530824     8.658878e-01     0.520376       6.88
      50     0.2400       166.362626     3.408579e-02     0.020485      25.40
     100     0.1200       166.388195     8.517242e-03     0.005119       4.00
    1000     0.0120       166.396627     8.515855e-05     0.000051     100.02

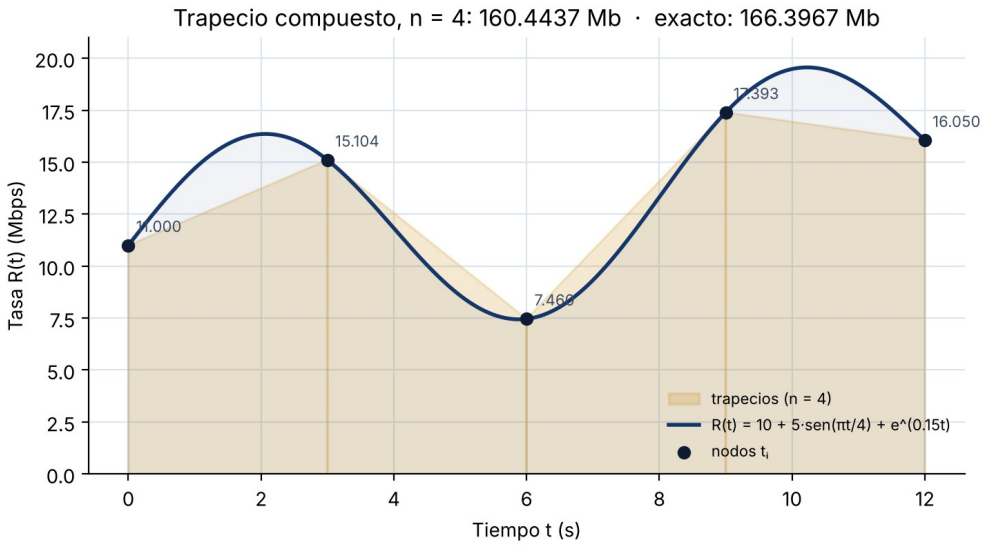
  'razón' = error anterior / error actual. El trapecio es de orden h²:
  si n se multiplica por k, el error se divide aproximadamente entre k².
  De 100 a 1000 (k = 10) el error baja unas 100 veces.

  Gráficas guardadas en trapecio_s12.png y convergencia_s12.png
```

Interfaz gráfica



Aplicativo web: ancho de paso, malla con pesos, fórmula con los números reemplazados, resultado frente al valor exacto y trapecios sobre la curva.



Gráfica generada por el programa: R(t), los cuatro trapecios de n = 4 y los nodos t_i.

6. Análisis de resultados

Exactitud con n = 4. Un error del 3.6 % es alto para dimensionar infraestructura. La causa es que h = 3 s es comparable con el periodo de la oscilación (8 s): con sólo dos nodos por media onda, las rectas no siguen la curvatura del seno.

Cota teórica. El error del trapecio compuesto cumple $|E| \le (b-a)\cdot h^2/12 \cdot \max|R''(t)|$, con $R''(t) = -(5\pi^2/16)\cdot\text{sen}(\pi t/4) + 0.0225\cdot e^{0.15t}$. En [0, 12] el máximo de $|R''|$ es ≈ 3.1396 (en t ≈ 6), así que con n = 4 la cota

es 12·9/12·3.1396 ≈ 28.26 Mb, que efectivamente es mayor que el error real de 5.95 Mb. Para garantizar un error menor que 0.01 Mb harían falta n ≥ √(12³·3.1396/(12·0.01)) ≈ 213 subintervalos.

Costo frente a precisión. Cada evaluación de R(t) equivale a una lectura del monitor de tráfico. Pasar de 4 a 100 subintervalos reduce el error de 5.95 a 0.0085 Mb (700 veces) con 96 lecturas más; en la práctica, un muestreo cada 0.12 s (n = 100) ya deja el error por debajo de 0.01 Mb (0.005 %).

7. Conclusiones

Se aplicó la regla del trapecio compuesto con n = 4 (h = 3 s) a la tasa de transferencia R(t) = 10 + 5·sen(πt/4) + e^(0.15t) y se obtuvo un total de 160.4437 Mb transferidos en 12 segundos, frente al valor exacto de 166.3967 Mb (error del 3.58 %). Se programó la función trapecio_compuesto(f, a, b, n) en Python y en un aplicativo web, y la tabla de convergencia para n = 4, 10, 50, 100 y 1000 mostró que el error cae proporcionalmente a h²: con n = 1000 el resultado, 166.396627 Mb, difiere del exacto en menos de una diezmilésima de Megabit.
